

ANN: Regression + Classification vs Classical ML

Scaler School of Technology | Deep Learning I | Week 8 — Session 1/5 | Post-Lecture Review & Practice



Key Takeaways

1 Classical ML dominates on small tabular data

MLPs rarely outperform Random Forest or SVM on structured data with fewer than 10K rows. Tree-based models handle non-linearity, mixed feature types, and missing values gracefully without normalization. *Always establish classical baselines before reaching for deep learning.*

2 MLP training loop: the five essential steps

Every PyTorch training loop follows: (1) **preprocess** with StandardScaler (zero mean, unit variance), (2) **define model** with hidden layers + activation, (3) **choose loss** (MSELoss for regression, BCELoss for binary classification), (4) **optimize** with Adam, (5) **monitor** train/val curves to catch overfitting.

3 Regularization prevents overfitting on small datasets

Two primary tools: **Dropout** (randomly zeros activations during training, reducing neuron co-adaptation), **L2 weight decay** (penalizes large weights via $\mathcal{L}_{reg} = \mathcal{L} + \lambda \sum_i w_i^2$, passed as `weight_decay` in Adam). Both reduce generalization gap on small tabular datasets.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why must we apply StandardScaler *before* feeding tabular data into an MLP? What breaks if we skip this step?
- Q2.** Your MLP achieves 95% training accuracy but only 70% validation accuracy. Which regularization technique would you try first, and why?
- Q3.** Random Forest achieves $R^2 = 0.82$ and your MLP achieves $R^2 = 0.79$ on California Housing. Does this mean the MLP is useless? Explain your reasoning considering practical trade-offs.
- Q4.** BCELoss requires model outputs in $(0, 1)$. Which activation function achieves this, and where in the architecture should it be placed?
- Q5.** Your MLP training loss decreases smoothly but validation loss oscillates wildly. What are two likely causes and how would you diagnose each?

Coding Exercises

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned.

Exercise 1: MLP Regressor with 3 Hidden Layers

Build a PyTorch MLP for California Housing with architecture: $[8 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1]$, ReLU activations, trained with Adam for 200 epochs. Plot train/val loss curves. Compute MSE and R^2 on the test set. Compare against `sklearn.neural_network.MLPRegressor` with the same topology. *Expected output:* dual-panel loss plot + metrics comparison table.

Exercise 2: ROC Curve Analysis for Heart Disease

Train your Heart Disease MLP. Save predicted probabilities (sigmoid output before thresholding). Using `sklearn.metrics.roc_curve` and `auc`, plot ROC curves for MLP, Logistic Regression, and SVM on the same axes. Mark the operating point at $\text{threshold} = 0.5$ with a dot. Include AUC values in the legend. *Expected output:* multi-model ROC plot with $\text{AUC} \geq 0.75$ for at least two models.

Exercise 3: Regularization Ablation Study

Train 4 variants of your classification MLP: (a) no regularization, (b) L2 only (`weight_decay=1e-4`), (c) Dropout(0.3) only, (d) both combined. Plot validation accuracy over epochs for all 4 on the same graph. Which combination gives the best generalization? Does regularization always help? *Expected output:* 4-line comparison plot + 1-paragraph written analysis.

Challenge Problem

When Does Deep Learning Actually Win? — Use MNIST to find a dataset where MLPs beat classical ML:

1. Load MNIST from `torchvision.datasets.MNIST` and flatten images to 784-dim vectors
2. Train Random Forest and SVM on 10,000 samples (Colab CPU, ~2 min each)
3. Train a 3-layer MLP $[784 \rightarrow 256 \rightarrow 128 \rightarrow 10]$ on the full 60,000 samples
4. Compare test accuracy across all three models
5. Repeat with only 1,000 training samples — does the ranking change?

Bonus: Add a single convolutional layer before the MLP. How much does accuracy improve?

Further Reading

- **Classical ML Foundation:** Hastie, Tibshirani & Friedman (2009), “*The Elements of Statistical Learning*” Ch. 10 — Random Forests, boosting, and when they beat neural nets.
- **DL Regularization:** Goodfellow, Bengio & Courville (2016), “*Deep Learning*” Ch. 7 — covers Dropout, L1/L2, early stopping, and data augmentation.
- **Tabular DL:** Grinsztajn et al. (2022), “*Why Tree-based Models Still Outperform Deep Learning on Tabular Data*” — rigorous benchmarks across 45 datasets.
- **Interpretability:** Lundberg & Lee (2017), “*A Unified Approach to Interpreting Model Predictions*” (SHAP) — understand why your model made a prediction.

Online Resources & Tools

- **PyTorch nn.Module** — official docs
- **Weights & Biases** — experiment tracking
- **scikit-learn User Guide** — all classical ML
- **Optuna** — automated hyperparameter tuning
- **FastAI tabular tutorial** — production tabular DL
- **Papers With Code** — tabular leaderboards

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about R^2 score?** StatQuest “*R-squared, Clearly Explained*” on YouTube — intuitive visual explanation of coefficient of determination.
- **Unsure when to use BCE vs CrossEntropy?** Revisit Week 2 Session 3 loss functions — BCE for binary outputs (sigmoid), CrossEntropy for multi-class (softmax).
- **Shaky on gradient descent?** Revisit Week 2 Session 1 backpropagation notebook — the chain rule and update rule underpin all PyTorch training.
- **Confused about train/val/test splits?** Fast.ai Lesson 1 covers the purpose of each split — validation for tuning, test for final unbiased evaluation.

Answers — Test Your Understanding

- A1.** Neural networks compute weighted sums. If feature A ranges 0–100,000 and feature B ranges 0–1, gradients for A dominate updates, causing slow or biased convergence. StandardScaler normalizes each feature to zero mean and unit variance, ensuring balanced gradient updates across all inputs.
- A2.** Dropout (e.g., rate = 0.3) is the first tool for overfitting MLPs. It randomly zeros activations during training, preventing neurons from co-adapting. A 25-point gap between train and val accuracy strongly suggests memorization rather than generalization, which Dropout directly addresses.
- A3.** Not necessarily. A 3% gap may not justify added complexity: DL requires careful tuning, more training time, and is harder to interpret. If $R^2 = 0.82$ meets business requirements, prefer the simpler model. DL becomes worthwhile when data scales to 100K+ samples or when interactions are too complex for tree splits.
- A4.** The sigmoid activation $\sigma(x) = 1/(1 + e^{-x})$ squashes any real value to (0,1). It should be the activation of the *final output neuron only* — hidden layers use ReLU. BCELoss expects probabilities and computes $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$.
- A5.** (1) *Learning rate too high:* gradients overshoot the minimum causing oscillations — diagnose by reducing LR by 10×. (2) *Batch size too small:* noisy gradient estimates per batch cause unstable updates — increase batch size or use gradient clipping (clip_grad_norm_).

Deep Dive: The Bias-Variance Tradeoff

The expected prediction error of a model decomposes as:

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}$$

High Bias (Underfitting):

- Model is too simple for the data
- Both train and val error are high
- Fix: more capacity (layers, units)
- Example: Linear model on non-linear data

High Variance (Overfitting):

- Model memorises training data
- Low train error, high val error
- Fix: regularisation, more data
- Example: Deep MLP on 300 samples

Classical ML models like Random Forests reduce variance through *bagging* (averaging many decorrelated trees), which is why they generalise well on small datasets without tuning.

Deep Dive: L1 vs L2 Regularization

Both penalties are added to the training loss $\mathcal{L}$:

L2 (Ridge / Weight Decay):

$$\mathcal{L}_{L2} = \mathcal{L} + \lambda \sum_i w_i^2$$

- Shrinks all weights uniformly
- Keeps all features active
- Smooth, differentiable penalty
- PyTorch: `Adam(weight_decay=\lambda)`

L1 (Lasso):

$$\mathcal{L}_{L1} = \mathcal{L} + \lambda \sum_i |w_i|$$

- Promotes **sparse** solutions
- Drives irrelevant weights to zero
- Non-differentiable at $w = 0$
- PyTorch: manual penalty term

When to use which: Prefer L2 for MLPs (smooth gradients, easy to implement). Use L1 when you suspect many irrelevant features and want automatic feature selection (e.g., in linear models via Lasso).

Deep Dive: Understanding AUC-ROC

The ROC curve plots **True Positive Rate** ($\text{TPR} = \text{TP}/(\text{TP} + \text{FN})$) vs **False Positive Rate** ($\text{FPR} = \text{FP}/(\text{FP} + \text{TN})$) at every decision threshold $\tau \in [0, 1]$.

$$\text{AUC} = P(\text{score}(\text{positive}) > \text{score}(\text{negative}))$$

- AUC = 0.5: model is random (diagonal line)
- AUC = 1.0: perfect separation
- AUC = 0.85: good; $\text{AUC} \geq 0.90$: excellent in practice

AUC is preferred over accuracy on **imbalanced datasets**: with 95% negative class, a model predicting “always negative” achieves 95% accuracy but $\text{AUC} = 0.5$.

Guided Experiments

Run these experiments in order and record your findings in a table:

1. **Width sweep:** Train MLPs with hidden sizes [8, 16, 32, 64, 128, 256, 512] (single layer). Plot validation accuracy vs. width. *Hypothesis:* accuracy plateaus above a threshold width.
2. **Optimizer comparison:** Train the same MLP architecture with SGD, Adam, and RMSprop. Plot loss curves on the same axes. *Hypothesis:* Adam converges faster on tabular data.
3. **Batch normalization:** Insert `nn.BatchNorm1d` between every hidden layer. Compare convergence speed and final validation accuracy vs. the baseline. Does BatchNorm help on tabular data?
4. **Depth ablation:** Train 1-layer, 2-layer, 3-layer, and 4-layer MLPs (all width 64). Plot test performance vs. depth. *Hypothesis:* more depth helps until overfitting dominates.

Mini-Project: Tabular DL on Kaggle

A guided end-to-end ML pipeline:

1. Download the **Titanic** dataset from Kaggle (`train.csv`, `test.csv`)
2. Handle missing values: median imputation for Age, mode for Embarked, drop Cabin
3. Encode categoricals: one-hot for Sex, Embarked; ordinal for Pclass
4. Feature engineer: `FamilySize = SibSp + Parch + 1`, IsAlone flag
5. Build and tune an MLP with 5-fold cross-validation
6. Submit predictions to Kaggle — target: **beat 80% accuracy**

Hint: The Titanic dataset is small (~890 rows) — Random Forest will likely tie or beat your MLP. This is by design; use the exercise to practice the pipeline.

Connect to the Bigger Picture

The PyTorch training loop you wrote in Session 1 — `zero_grad` → `forward` → `loss` → `backward` → `step` — is **identical** in every subsequent session. Only the architecture and loss function change:

Session	Architecture	Loss
1 (this session)	MLP (tabular)	MSELoss / BCELoss
2 (Style Transfer)	VGG19 feature extractor	Content + Style + TV
3 (Shakespeare)	CharLSTM	CrossEntropyLoss
4 (Translation)	Seq2Seq + Attention	CrossEntropyLoss
5 (Transformer)	Full Transformer	CrossEntropyLoss

The debugging skills you practiced here (loss curves, overfitting detection, regularization) are universally applicable.

Model Comparison Quick Reference

Model	Best For	Key Hyperparameters	Sklearn API
Linear Regression	Low-noise, linear relationships	fit_intercept	LinearRegression()
Random Forest	Tabular, non-linear, mixed types	n_estimators, max_depth	RandomForestRegressor()
SVM	Small datasets, clear margins	C, kernel, gamma	SVC(probability=True)
MLP (sklearn)	Quick baseline DL	hidden_layer_sizes, alpha	MLPClassifier()
MLP (PyTorch)	Custom architectures, GPU	lr, weight_decay, dropout	nn.Sequential()

PyTorch Training Loop Template

```

for epoch in range(num_epochs):
    model.train()
    optimizer.zero_grad() # 1. Clear accumulated gradients
    y_pred = model(X_train) # 2. Forward pass
    loss = criterion(y_pred, y_train) # 3. Compute loss
    loss.backward() # 4. Backpropagate gradients
    optimizer.step() # 5. Update weights

# Validation (no gradient tracking)
model.eval()
with torch.no_grad():
    val_pred = model(X_val)
    val_loss = criterion(val_pred, y_val)

```

Debugging Checklist

Symptom	Likely Cause	Fix
Loss not decreasing	LR too small, or wrong loss function	Increase LR $\times 10$; check criterion matches task
Loss exploding (NaN)	LR too large, or extreme input values	Reduce LR; add clip_grad_norm_; check StandardScaler
Accuracy stuck at 50%	Class imbalance or label encoding error	Check class distribution; verify labels are 0/1 not 1/2
Train much better than val	Overfitting	Add Dropout; reduce model size; get more data
Val loss decreasing then rising	Late overfitting	Use early stopping or reduce LR on plateau

Hands-on notebook: Session_1_ANN_Regression_Classification.ipynb

Instructor: Priyansh Saxena